Due: 09.11.2026 23:55

CENG211 – Programming Fundamentals

HOMEWORK 1
EVENT TICKET BOOKING SYSTEM

Concepts:

- Classes and Objects
- Constructors
- Encapsulation
- Inheritance
- Arrays and ArrayList
- CSV File I/O
- Object-Oriented Design.

1. Introduction

An event management company organizes concerts, conferences, and sports events. Develop a Java Event Ticket Booking System that reads the provided CSV files, manages customers and events, performs ticket bookings and cancellations, applies a group-specific booking policy, and answers the required queries.

Each event has a limited capacity and a fixed ticket price. Customers may book multiple tickets if sufficient seats and balance are available. Premium customers receive a 5% discount on successful purchases. Each group implements one additional booking policy determined by its group number.

You must decide the appropriate attributes, data types, constructors, methods, parameters, and return types. Do not expect starter code or predefined method signatures.

2. System Description

2.1 Input Files

Read the two supplied CSV files. Do not hardcode event or customer records.

events.csv field	Meaning
Event ID	Unique event identifier
Event Name	Event name
Event Category	One of CONCERT, CONFERENCE, SPORTS
Capacity	Maximum number of tickets
Ticket Price	Price per ticket

customers.csv field	Meaning
Customer ID	Unique customer identifier
Customer Name	Customer name
Balance	Available balance
Membership Type	REGULAR or PREMIUM

Event categories and membership types are fixed predefined sets. Values outside these sets must not be assignable.

2.2 Customers and Discounts

A premium customer is a customer, but not every customer is a premium customer. A customer can be either regular or premium. Common information and operations must not be unnecessarily duplicated. Premium customers receive an additional 5% discount on ticket purchases. When a group-specific purchase discount also applies, calculate that discount first, then apply the premium discount to the remaining amount.

Premium final price = price × 0.95. With a 10% group discount and premium membership: final price = original price × 0.90 × 0.95.

2.3 Bookings and Capacity

Each booking must identify its booking, customer, event, ticket quantity, original total price, final paid amount, and status. Booking status can only be ACTIVE or CANCELLED. Each successful booking receives a unique ID.

A booking succeeds only when the customer and event exist, ticket quantity is positive, enough seats remain, the customer can pay the final price, and the enabled group-specific rule allows the transaction. Successful bookings deduct the payment and reserve seats; failed bookings create no booking and change no state.

Available seats = event capacity − tickets in ACTIVE bookings for that event. Only active bookings count toward occupied seats.

2.4 Cancellations

Only an existing ACTIVE booking can be cancelled. Cancellation releases its tickets, updates its status to CANCELLED, and credits the refund to the customer. A cancelled booking remains in the system and cannot be cancelled again. The default refund is the final amount paid; the CANCELLATION_FEE policy changes this when enabled. A failed cancellation must not change the system state.

3. Required Class Design

Your project must include the classes below. You may add helper classes. The descriptions state responsibilities, not ready-made Java fields or method signatures.

Class	Responsibility
Event	Maintain event identification, category, capacity, and ticket price.
Customer	Maintain regular customer identification and balance.
PremiumCustomer	Inherit from Customer and support the premium membership benefit.
Booking	Maintain reservation details, paid amount, and status.

BookingSystem	Manage events, customers, bookings, reservations, cancellations, available seats, and booking policies.
FileIO	Load events and customers from CSV files.
Query	Answer the four required queries.
TicketBookingApp	Contain main and execute the required operations in order.

Use ArrayList collections for event, customer, and booking records. The three predefined sets—event categories, booking statuses, and booking policies—must use appropriate Java types that prevent arbitrary values.

4. Group-Specific Booking Policies

Let G be your assigned group number. Determine the policy by RuleNumber = ((G − 1) % 5) + 1. Define the group number once; calculate the selected policy and its parameter programmatically. Policy names form a fixed predefined set.

Rule number	Policy
1	BOOKING_LIMIT
2	GROUP_DISCOUNT
3	CATEGORY_RESTRICTION
4	CANCELLATION_FEE
5	RESERVED_CAPACITY

Rule 1 — BOOKING_LIMIT

MaximumTickets = 2 + (G % 3). A customer may hold no more than this number of ACTIVE tickets for the same event across all their bookings. Cancelled tickets do not count. This applies to both membership types.

Rule 2 — GROUP_DISCOUNT

DiscountRate (%) = 5 + 5 × (G % 3). Apply this discount only when at least four tickets are purchased in ONE booking. Prior bookings do not count. Apply the premium 5% discount afterward if applicable.

Rule 3 — CATEGORY_RESTRICTION

Restrict one event category to premium customers only. Regular customers may book other categories as usual.

G % 3	Restricted category
0	CONCERT
1	SPORTS
2	CONFERENCE

Rule 4 — CANCELLATION_FEE

CancellationFeeRate (%) = 5 + 5 × (G % 3). Cancellation fee = final amount originally paid × rate / 100. Refund = final amount paid − cancellation fee. Apply to both membership types. Other policies use full refunds.

Rule 5 — RESERVED_CAPACITY

ReservedPercentage (%) = 10 + 5 × (G % 3). ReservedSeats = floor(capacity × ReservedPercentage / 100). RegularSeatLimit = capacity − ReservedSeats. The total ACTIVE tickets held by regular customers for an event cannot exceed RegularSeatLimit. Premium customers may book any remaining seats, but the overall capacity cannot be exceeded. Cancelled tickets do not count.

5. Required Program Execution

Run the following sections automatically and in this exact order: (1) System Information, (2) Common Operations, (3) Queries, (4) Group-Specific Rule. Do not prompt for user input or create an interactive menu.

Phase 1 — Basic System: Load the supplied CSV files. Execute common operations with the group-specific policy DISABLED; general validations and the 5% premium discount still apply. Run queries after common operations. The first three output sections must be identical for all groups.

Phase 2 — Group-Specific System: Create a NEW instance of the SAME booking system implementation, reload the original CSV data, and start with original balances, full event capacities, and no bookings. Determine and ENABLE your group policy. Execute only the corresponding rule demonstration in Section 7. Do not create two separate implementations of BookingSystem.

6. Common Operations and Queries

6.1 System Information

Print the total event count, customer count, and premium customer count.

```
========== SYSTEM INFORMATION ==========
Events: ...
Customers: ...
Premium Customers: ...
```

6.2 Common Booking and Cancellation Operations

Perform B1–B6 in the exact order below. These IDs refer to customers and events loaded from the CSV files; the operation labels B1–B6 are not CSV records. Do not display expected results from a hardcoded answer key.

Operation	Customer ID	Event ID	Tickets
B1	201	101	1
B2	202	103	1
B3	205	104	6
B4	203	101	1
B5	201	102	0
B6	205	102	2

For each booking, print SUCCESS and the final payment, or FAILED and its reason. Then perform C1: cancel the booking created by B1; C2: attempt to cancel the same booking again. Print SUCCESS and refund, or FAILED and reason.

Use these failure labels when applicable: Insufficient Capacity; Insufficient Balance; Invalid Quantity; Customer Not Found; Event Not Found; Already Cancelled.

```
========== COMMON OPERATIONS ==========
B1: <RESULT> | <Paid: AMOUNT or FAILURE REASON>
...
B6: <RESULT> | <Paid: AMOUNT or FAILURE REASON>
C1: <RESULT> | <Refund: AMOUNT or FAILURE REASON>
C2: <RESULT> | <Refund: AMOUNT or FAILURE REASON>
```

6.3 Queries

Execute these queries after C2. Only ACTIVE bookings count. Revenue is the sum of final paid amounts for active bookings (cancelled bookings are excluded). In a tie, choose the record with the smallest ID.

1. Most Popular Event: event name and number of active reserved tickets.
2. Highest Revenue Event: event name and total revenue from active bookings.
3. Top Customer: customer name, membership type, and total number of active reserved tickets across all events.
4. Tickets by Category: total active reserved tickets for CONCERT, CONFERENCE, and SPORTS, in that order.

If there are no active bookings, print an appropriate message or zero totals.

```
========== QUERIES ==========
Q1: <EVENT NAME> | <COUNT> tickets
Q2: <EVENT NAME> | <REVENUE>
Q3: <CUSTOMER NAME> | <MEMBERSHIP> | <COUNT> tickets
Q4: CONCERT: <COUNT> | CONFERENCE: <COUNT> | SPORTS: <COUNT>
```

7. Group-Specific Rule Demonstration

Use the fresh system created in Phase 2. Print your group number, the actual selected policy, and its calculated parameter. Then run ONLY the scenario for your assigned policy. All R-operation results must come from actual booking/cancellation logic.

```
========== GROUP-SPECIFIC RULE ==========
Group: <GROUP NUMBER>
Rule: <SELECTED POLICY>
Parameter: <CALCULATED VALUE>
```

7.1 BOOKING_LIMIT — Groups 1, 6, 11, ...

Use customer 201 and event 105. Let L be the maximum allowed active ticket count calculated for your group.

5. R1: Book L tickets.
6. R2: Attempt to book one additional ticket for the same customer and event.
7. R3: Cancel the booking created by R1.
8. R4: Attempt to book one ticket again.

For each operation, print the result and payment/refund amount or failure reason.

7.2 GROUP_DISCOUNT — Groups 2, 7, 12, ...

Use event 101. R1: Customer 201 books 3 tickets. R2: Customer 202 books 4 tickets. For each, print result, original total, group discount amount, premium discount amount, and final payment.

7.3 CATEGORY_RESTRICTION — Groups 3, 8, 13, ...

Determine the restricted category and select its event ID from the table. R1: Customer 201 attempts to book 1 ticket. R2: Customer 202 attempts to book 1 ticket for the same event. Print restricted category, selected event, each result, and payment or failure reason.

Restricted category	Event ID
CONCERT	101
CONFERENCE	102
SPORTS	103

7.4 CANCELLATION_FEE — Groups 4, 9, 14, ...

Use customer 201 and event 102. R1: Book 2 tickets. R2: Cancel the booking created by R1. Print booking result, original payment, cancellation fee percentage and amount, refund, and cancellation result.

7.5 RESERVED_CAPACITY — Groups 5, 10, 15, ...

Use event 106. Let R be the calculated RegularSeatLimit for your group. R1: Customer 201 books R tickets. R2: Customer 201 attempts to book 1 additional ticket. R3: Customer 202 attempts to book 1 ticket. Print reserved seat count, each result and payment or failure reason, and available seats after the operations.

8. Output and Implementation Requirements

- Follow the four output sections and the specified operation order exactly. Do not print long debug logs in the final submission.
- Use two decimal places for all monetary values.
- All displayed results must be calculated from actual objects and operations, not hardcoded output strings.
- Use relative CSV file paths and standard Java libraries only.
- A failed booking or cancellation must leave all system data unchanged.
- Keep cancelled bookings for history, but exclude them from active ticket counts and query revenue.
- The final payment must be computed before checking the customer's available balance.
- Use the same booking implementation in both phases; only the group policy enablement and initial data differ.
- You are responsible for your own attributes, constructors, methods, parameters, and return types.

10. Submission Rules

9. Complete the homework in your assigned group of 3–4 students. Only one member submits.
10. The project must be executable in Eclipse and submitted via MS Teams by the deadline.
11. Name the project GXX_CENG211_HW1 (replace XX with your group number).
12. Submit a ZIP file with the same name, e.g., G07_CENG211_HW1.zip.
13. Include all Java source files and the required CSV files. Do not submit only compiled .class files.

14. Academic integrity rules apply. Submit your group's own work.